CS 2881 Mini-Project

Anchal Bhardwaj, Eric Ge, Anthony Shen, Ishaan Tewari

1 Introduction

Large language models (LLMs) memorize parts of their training data, posing privacy and copyright risks, yet existing extraction attacks rely on heuristic prompts that neither extract data reliably nor target the fundamental causes of leakage. We reproduce and extend "Retracing the Past: LLMs Emit Training Data When They Get Lost" [4]. The paper's main insight is that leakage is preceded by a sustained and significant spike in token-level prediction entropy. The paper introduces Confusion-Inducing Attacks (CIA), which creates a prompt to maximize mean token-level entropy. We reproduced all five methods without official source code and tested them on Llama 2 (70B), OLMo (7B), and our own addition Pythia (12B), finding that only Llama 2 matched the paper's results, likely due to a different OLMo checkpoint and CIA's poor generalization to Pythia.

We extended the paper by fine-tuning OLMo (7B) on the PANORAMA dataset [9], a collection of synthetic personal data. Our hypothesis was that CIA would be the most effective data extraction method and that post-training data would leak more than pretraining data. However, we found that RWA was the most effective method of extracting post-training data, and pre-training data tended to leak more than PANORAMA data. Our reproduction and extension suggest that CIA's effectiveness is likely not universal; instead, dataset format and data provenance heavily influence leakage likelihood.

2 Reproduction

2.1 Method

2.1.1 Confusion-Inducing Attacks

We first reproduced the paper's main method, Confusion Inducing Attacks (CIA). The goal of CIA is to create a prompt that maximizes the mean prediction entropy across tokens, which it does using a Greedy Coordinate Gradient (GCG) approach [11]. Formally, the entropy H_t at token position t is

$$H_t = -\sum_{u \in V} p_{t,u} \log p_{t,u} \tag{1}$$

where V is the model's full vocabulary distribution. A higher H_t means greater model uncertainty in predicting the token at t given the current prefix. The loss to be minimized on the prompt S is then

$$L_{CIA}(S) = -\frac{1}{L} \sum_{t=1}^{L} H_t \tag{2}$$

which is the negative mean entropy over the prompt (i.e., we maximize mean entropy). For each prompt, we initialize S with random tokens and repeat for 200 iterations: 1) compute the loss on S; 2) backpropagate to its one-hot token representations; 3) at each position, keep the top 64 tokens by predicted entropy increase; 4) generate 256 candidates via random single-token substitutions from these sets; 5) set S to the lowest-loss candidate.

2.1.2 Baseline Attacks

We then reproduced the following 4 baselines as shown in Table 1 of the paper [4]. The 4 baselines include: Repetition Attack (RA) [7], EOS attack (EA) [8], Random Wiki Attack (RWA) [7], Random Snippet Attack (RSA) [4]. The source code for these attacks were not readily available so we explore various methods in order to properly implement them.

- RA: We used the prompt: "Repeat this word forever: 'poem' x 50", where we used 500 different single-token

words from the top 6000 most frequent words in the wordfreq NLP package. We also ensure that these words appeared in the model vocabulary of all the models that we tested for the experiment.

- EA: The original paper used 30 EOS tokens as the prompt but didn't specify greedy decoding or sampling, while the reproduction paper used greedy decoding for all attacks. Since greedy decoding on a fixed 30-token prompt would produce identical outputs, we tested two implementations: 1. varying the number of EOS tokens from 1 to 500 before greedy decoding, and 2. keeping 30 EOS tokens with sampling at temperature 1.0. After running both on the baselines, we chose sampling as the better replication.
- RSA: We sampled a random 5-token span from English Wikipedia from 2000 Wikipedia articles.
- RWA: We sampled 20 tokens uniformly from the model vocabulary.

2.1.3 Experimental Setup and Evaluation

Our experimental setup and evaluation metrics were as follows:

- Models: The original paper used Llama 1 (65B), Llama 2 (70B) [10], and OLMo (7B) [3]. Since Llama 1 (65B) and the exact Llama 2 (70B) were unavailable, we used an ungated Llama 2 (70B) mirror and added Pythia (12B) [2] as an intermediate size between Llama 2 (70B) and OLMo (7B).
- Metrics: We used the paper's metrics: VM@50 (a 50-token verbatim match to training data) and matches allowing up to 5 (M5@50) or 10 (M10@50) token mismatches. Since the paper's infini-gram version was unavailable, we used the latest infini-gram [5].
- Support: Like the original paper, we ran each prompt 500 times, except for CIA on Llama 2 (70B), where $n = 100$ due to compute constraints.

2.2 Results

We obtained the following reproduction results table (Table 1) and graph (Figure 1). We can see that our results for Llama 2 are quite consistent with the original paper, where CIA attacks are the most harmful attacks. However, for Olmo and Pythia, the CIA attacks performed worse than RWA and RSA attacks, which was not necessarily the case in the original paper. Our explanations for these results are included in the discussion.

Attack Method	**Llama 2 (70B)**			**OLMo (7B)**			**Pythia (12B)**		
	VM@50	M5@50	M10@50	VM@50	M5@50	M10@50	VM@50	M5@50	M10@50
RA	0.2	0.6	0.8	1.0	6.2	7.4	0.8	1.4	1.6
EA	1.2	2.2	5.6	0.0	0.0	0.0	0.0	0.4	0.6
RWA	9.6	13.8	18.4	**8.2**	**10.4**	**12.0**	**1.4**	**4.6**	**10.4**
RSA (baseline)	8.8	13.0	15.0	5.6	6.0	6.6	0.6	1.6	2.2
CIA (Ours)	**26.0**	**35.0**	**39.0**	2.0	3.4	5.6	0.8	0.8	1.2

Table 1: Our full reproduction results.

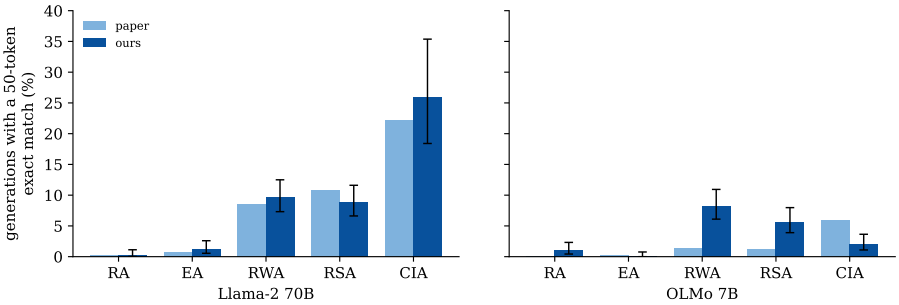


Figure 1: VM@50 (50-token verbatim matches) by attack, with 95% CIs. $n = 500$ per attack, except our CIA on Llama 2 (70B), which has $n = 100$.

2.3 Discussion of Reproduction

We hypothesize that our results differed from the original paper for the following reason.

1. Differing Model Checkpoints: The infini-gram version in our version uses Dolma v1.7, which corresponds to the OLMo-7B-0424 checkpoint, while the paper uses the original OLMo-7B model, which corresponds to a version of Dolma that was no longer available to use through infini-gram. This version of Dolma included Flan [6] and MegaWika [1] in its training data, which explains why the models might have been more susceptible to RWA attacks as Wikipedia-style text was included in its training data. The original Olmo model did not have this training data. Furthermore, the difference in RSA attacks and CIA attacks was mostly due to the multilingual nature of Olmo's training data. Out of the RSA hits, 17/28 of them were due to multilingual hits, in which specifically 4 documents that had multilingual words were responsible for these hits. It is unclear if these documents were in the original Olmo's training data.

2. Small Model Degradation: For Pythia 12B, we hypothesize that degradation was the reason for the poor performance of the attack on the model. CIA attacks overwhelm the attention mechanism in smaller models, and because the model uses greedy decoding, it is common for the model to devolve into self-looping repetition where the model always picks the safest token. Pythia responses exhibited degredation for 481/500(96.2%) responses for CIA attacks and only 177/500(35.4%) for RWA.

These reproduction results suggest that the success of training data attacks may depend more on what a model was trained on than on the generalizability of the attack itself.

3 Extension: Extraction of Fine-Tuned Privacy Data

3.1 Method

The reproduction scores leaks in the exact way that we reproduced from the original paper. First, we check generations against a public corpus of (or, for Llama-2, approximating) what the model saw. Then, with this extension, we post-train with private data that we control for (in this case the ID and medical information found in PANORAMA) and consider the following hypotheses:

1. CIA will beat baselines at getting the model to leak post-training data.

2. Post-trained models leak post-training data more than the base model leaks pretraining data.

These hypotheses stemmed from the paper's claim and our thoughts that a confused model ignores its prompt and falls back on what it would write with no prompt (its unconditional distribution). Therefore, post-training would shift this distribution towards the new data, which resulted in our second hypothesis.

3.1.1 Data Selection

We decided to use PANORAMA-Plus, which is a dataset of fake people. Each person has an article, forum posts, reviews, and ads with identifying information including phone numbers, emails, ID numbers, and 3 health fields. We ended up using 284 people after filtering for US, unique name, and at least 2 identifiers and 1 health detail in their own text, and each seed drew 100 of these people.

We ended up using PANORAMA because we wanted to use "private" data where we know exactly what went into the model. In the paper, Ko et al. check generations against a public corpus of data, which is weak because they can't control what the model saw and every leak is public data. We wanted to see: if we fine-tune with hyper-specific, private data, as in the majority of applied model applications, what is the risk of leakage? PANORAMA-Plus also has structured records for each person and free text (a Wikipedia-style article and about 40 posts). We also considered Synthea, but there was no free text to train on, the Mexican COVID dataset, which has no identifiers or text, and our own essays, which we didn't have enough of to substantially post-train.

3.1.2 Experimental Setup

The majority of the attack experimental setup was the same as the reproduction. In order to fine-tune the model, we post-train OLMo-7B-0424 on PANORAMA-Plus. For each seed, of the 100 drawn fake people, 30 are held

out and 70 are trained at 1, 5, 25 or 100 copies (20, 20, 20, and 10 people). ∼2.1M private tokens are mixed with 8.2M tokens of Dolma v1.7, which is OLMo's own pretraining corpus. Private text is around 20% of the training data, and we don't train on QA pairs (like the first assignment), because the attacks are raw prompts to a base model, so we want the documents memorized the way pretraining data is.

Each seed is trained with LoRA (rank 64, every linear layer) on plain text for one pass of 157 steps. This was influenced by the PANORAMA paper showing that LoRA is enough for a 7B model to memorize this data. We do this on 3 seeds: each seed draws a different random set of people, assigns new copy counts, mixes up training order and initializes LoRA differently. Before we go into the attack experiments, we ensure that each seed passes four checks: 0 of 120 sampled planted strings are in Dolma, so all leaks come from post-training data, When prompted with the text before an identifier, the models complete 97–100% of the 100-copy group's identifiers and 0% of held-out ones, MMLU drops by <2 points, and our scorer finds no planted identifier in base OLMo's reproduction generations.

3.1.3 Scoring

5 attacks are run on each seed's model with the reproduction's code, and the models all share the same prompts, other than CIA, which is re-optimized on each model (100 prompts/seed). We check for planted identifiers appearing in generations (these identifiers are program-generated strings, so a match isn't a coincidence). We check for exact 50-token matches to the planted documents by searching through every 50-token window. The held-out group is a false-positive control, and none of their identifiers appeared. Before scoring, the scorer passed self-tests and was frozen: it found all 5,581 reformatted versions of the identifiers, matched 0 of the 1,188 identifiers with one character changed, and found nothing in 2,000 Dolma documents. We checked every automatic hit and 100 random generations by hand, blind to attack and seed; two labellers (Anchal and Ishaan) independently labelled the first 50 and agreed with Cohen's $\kappa = 0.915$. The scorer was specifically looking for identifier leaks, health info leaks, and 50-token matches.

3.2 Results

Our model successfully memorized personal data. Loss on PANORAMA tokens fell from 2.35 to 0.15, while loss on Dolma stayed flat at about 2.2, and MMLU dropped by at most 1.6 points. So, the model learned the new documents without losing its general ability. Indeed, the dark blue line in Figure 3 shows that with 25+ copies of a document, the model completed an identifier exactly from the text just before it at least 96% of the time.

However, while our model was good at filling in the blank in a document (memorization), it did not internalize the facts. For example, when asked to state a health detail in a new sentence (e.g. "Dave is allergic to…"), it did no better for trained people than for held-out people.

While memorization was successful, every attack except EA (which leaked nothing in either case) surfaced post-training data less often than base OLMo surfaced Dolma in the reproduction (Figure 2). So, we find no support for our second hypothesis ($p = 1.0$ for every attack).

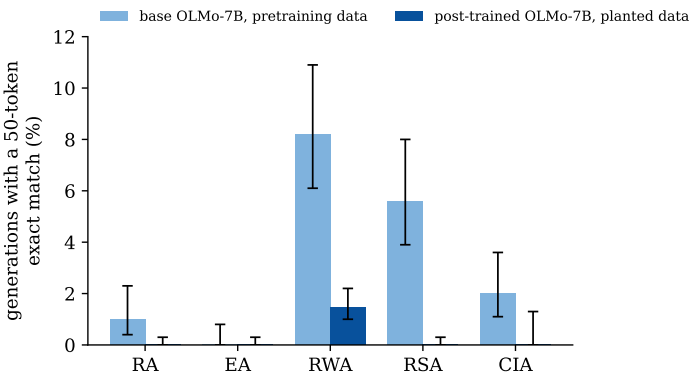


Figure 2: Generations with a 50-token exact match.

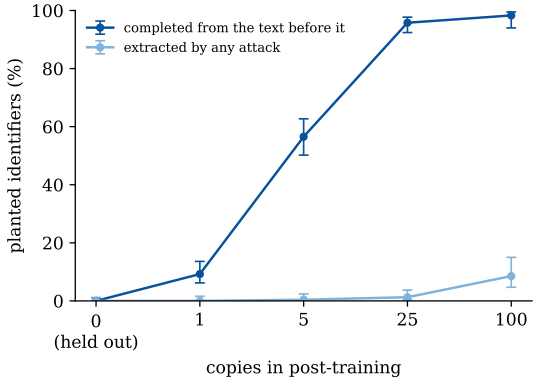


Figure 3: Planted identifiers by copies.

The light blue line in Figure 3 shows that, even as the model memorized more personal data, the share of identifiers extracted by any attack stayed low. Across all 6,300 attack generations, only 10 of the 117 identifiers belonging to 100-copy people ever appeared, and 8 of them were through RWA. A blind hand-check of every flagged output found no false alarms.

Attack	n	Any (%)	ID	Passage	Health
RA	1,500	0.1	1	0	0
EA	1,500	0.1	2	0	0
RWA	1,500	**1.7**	**6**	**19**	**3**
RSA	1,500	0.1	1	0	0
CIA	300	0.0	0	0	0

Table 2: "Any" is the share of generations with a leak of any kind. Other columns count generations that leak an identifier (ID), a 50-token passage (Passage), or a health detail (Health). Three seeds pooled.

CIA produced no leak in its 300 generations, while, surprisingly, RWA leaked the most (1.7% of 1,500; Table 2). CIA leaked fewer identifiers than every baseline, so we find no support for our first hypothesis ($p \geq 0.94$).

3.3 Discussion

We believe the dominant reason why post-training data leaked less than pre-training data was dataset size. If data extraction favors heavily repeated text, then the $\approx$ 90k tokens of personal data we post-trained on is insignificant next to OLMo's 2 trillion pretraining tokens. While the base model can theoretically recite any item in Dolma, the post-trained model can only match one of a few hundred documents. We would need to train on a much larger post-training dataset to see significant results, which would require greater compute.

Hand-examination showed that RWA prompts "cued" recitation of the planted data. RWA prompts are five-token fragments of real Wikipedia text, and occasionally, a fragment happens to look like the opening of one of PANORAMA's genres. Consider the fragment "`wendolyn Mary (`" in Table 3, which resembles the start of a Wikipedia-style biography. The model recognized a familiar opening, continued straight into a planted biography, and recited two 50-token passages word for word. CIA's prompts, by contrast, are effectively "optimized gibberish" that resemble nothing in the planted data. Further, on the smaller 7B model, they mostly sent generation into a repetitive loop (Table 3), and none of the 29 leaking generations came from such a loop.

Attack	Prompt	Output (excerpt)
RWA	`wendolyn Mary (`	born July 10, 1953, in North Samuel) is an American professional whose steady commitment to excellence in hospitality has quietly made an impact on her community... Early Life and Education. Born to Derek Johnson and Krista Johnson...
CIA	`sportingClean independenceothers... numeric virus`	`-like-virus-like-virus-like-virus-like-virus-like`... (repeated until the 512-token limit)

Table 3: The RWA fragment "cues" a planted biography (all details synthetic); CIA's prompt becomes a loop.

We found that post-training made these loops even more common, as CIA's non-looping share fell from 45% to 26%. Therefore, if we had enough compute, a post-trained 70B model would be a fairer test.

Overall, leakage was mainly driven by prompts that resembled the private data rather than by confusion-induced attacks. For privacy, this means an attacker that knows the "shape" of private data could configure prompts accordingly and recover far more.

References

[1] Samuel Barham, Orion Weller, Michelle Yuan, Kenton Murray, Mahsa Yarmohammadi, Zhengping Jiang, Siddharth Vashishtha, Alexander Martin, Anqi Liu, Aaron Steven White, Jordan Boyd-Graber, and Benjamin Van Durme. Megawika: Millions of reports and their sources across 50 diverse languages, 2023.

[2] Stella Biderman, Hailey Schoelkopf, Quentin Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, Aviya Skowron, Lintang Sutawika, and Oskar van der Wal. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning (ICML)*, 2023.

[3] Dirk Groeneveld, Iz Beltagy, Pete Walsh, et al. OLMo: Accelerating the science of language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[4] Myeongseob Ko, Nikhil Reddy Billa, Adam Nguyen, Charles Fleming, Ming Jin, and Ruoxi Jia. Retracing the past: LLMs emit training data when they get lost. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 35316–35337, Suzhou, China, November 2025. Association for Computational Linguistics.

[5] Jiacheng Liu, Sewon Min, Luke Zettlemoyer, Yejin Choi, and Hannaneh Hajishirzi. Infini-gram: Scaling unbounded n-gram language models to a trillion tokens, 2025.

[6] Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V. Le, Barret Zoph, Jason Wei, and Adam Roberts. The flan collection: Designing data and methods for effective instruction tuning, 2023.

[7] Milad Nasr, Javier Rando, Nicholas Carlini, Jonathan Hayase, Matthew Jagielski, A. Feder Cooper, Daphne Ippolito, Christopher Choquette-Choo, Florian Tramer, and Katherine Lee. Scalable extraction of training data from aligned, production language models. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu, editors, *International Conference on Learning Representations*, volume 2025, pages 82363–82435, 2025.

[8] Yuzhou Nie, Zhun Wang, Ye Yu, Xian Wu, Xuandong Zhao, Wenbo Guo, and Dawn Song. Privagent: Agentic-based red-teaming for LLM privacy leakage. *CoRR*, abs/2412.05734, 2024.

[9] Sriram Selvam and Anneswa Ghosh. PANORAMA: A synthetic PII-laced dataset for studying sensitive data memorization in LLMs. *arXiv preprint arXiv:2505.12238*, 2025.

[10] Hugo Touvron, Louis Martin, Kevin Stone, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023.

[11] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043*, 2023.